Divide-and-Conquer Technique: Solving Recurrences

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

How to Solve Recurrence Relations?

- Methods for solving recurrence relations:
  - iteratively expansion
  - using the recursion tree
  - Master Theorem

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

Solving Recurrences: Iterative Expansion

Finding Maximum and Minimum:

$$T(n) = \begin{cases} b & \text{if } n = 1 \text{ or } 2 \\ 2T(n/2) + b & \text{if } n > 2 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + b \\ &= 2(2T(n/2^2) + b) + b \\ &= 2^2 T(n/2^2) + 2b + b \\ &= 2^3 T(n/2^3) + 2^2 b + 2b + b \\ &= \ldots \\ &= 2^i T(n/2^i) + (2^{i-1} + 2^2 + 2 + 1)b \\ &= 2^i T(n/2^i) + (2^{i-1+1} - 1)b \end{aligned}$$

- We stop when $n/2^i = 1$, that is, $2^i = n$.
  Then, $i = \log_2 n$.
- So, $T(n) = bn + b(n-1) = 2bn - 1$
- Thus, $T(n) = O(n)$.

Solving Recurrences: Iterative Expansion

Merge Sort (both Best-case & Worst-case):

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + bn \\ &= 2(2T(n/2^2) + b(n/2)) + bn \\ &= 2^2 T(n/2^2) + 2bn \\ &= 2^3 T(n/2^3) + 3bn \\ &= 2^4 T(n/2^4) + 4bn \\ &= \ldots \\ &= 2^i T(n/2^i) + ibn \end{aligned}$$

- We stop when $n/2^i = 1$, that is, $2^i = n$.
  Then, $i = \log_2 n$.
- So, $T(n) = bn + bn \log n$
- Thus, $T(n) = O(n \log n)$.

Solving Recurrences: Iterative Expansion

Quick Sort (Best-case):

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + bn \\ &= 2(2T(n/2^2) + b(n/2)) + bn \\ &= 2^2 T(n/2^2) + 2bn \\ &= 2^3 T(n/2^3) + 3bn \\ &= 2^4 T(n/2^4) + 4bn \\ &= \ldots \\ &= 2^i T(n/2^i) + ibn \end{aligned}$$

- We stop when $n/2^i = 1$, that is, $2^i = n$.
  Then, $i = \log_2 n$.
- So, $T(n) = bn + bn \log n$
- Thus, $T(n) = O(n \log n)$.

Solving Recurrences: Iterative Expansion

Quick Sort (Worst-case):

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ T(n-1) + bn & \text{if } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= T(n-1) + bn \\ &= T(n-2) + b(n-1) + bn \\ &= T(n-3) + b(n-2) + b(n-1) + bn \\ &= \ldots \\ &= T(n-i) + b[(n-i+1) + \ldots + (n-2) + (n-1) + n] \end{aligned}$$

- We stop when $n - i = 1$. That is, $i = n - 1$. Then

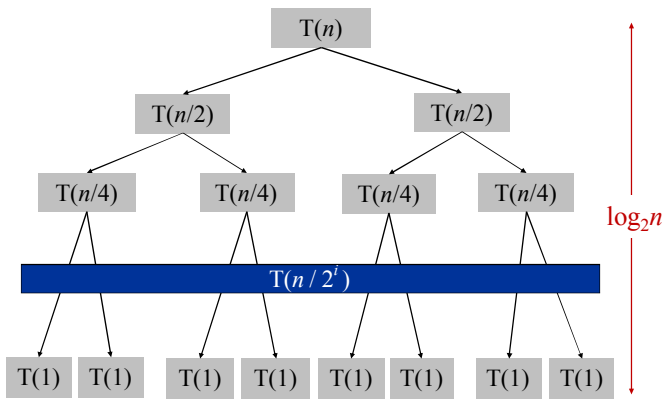
$$\begin{aligned} T(n) &= T(n-i) + b[(n-i+1) + \ldots + (n-2) + (n-1) + n] \\ &= T(1) + b[2 + \ldots + (n-2) + (n-1) + n] \\ &= b[1 + 2 + \ldots + (n-2) + (n-1) + n] \\ &= bn(n+1)/2 \end{aligned}$$

- Thus, $T(n) = O(n^2)$.

Solving Recurrences: Recursion Tree

Merge Sort (both Best-case & Worst-case):

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n > 1 \end{cases}$$



No. of Nodes at this Level	Time for Each Node	Time at this Level
1	bn	$1(bn) = bn$
2	$bn/2$	$2(bn/2) = bn$
4	$bn/4$	$4(bn/4) = bn$
2^i	$bn/2^i$	$2^i(bn/2^i) = bn$
n	b	$n(b) = bn$

$$T(n) = \underbrace{bn + bn + \ldots + bn}_{\log n \text{ times}} = bn\log n = O(n\log n)$$

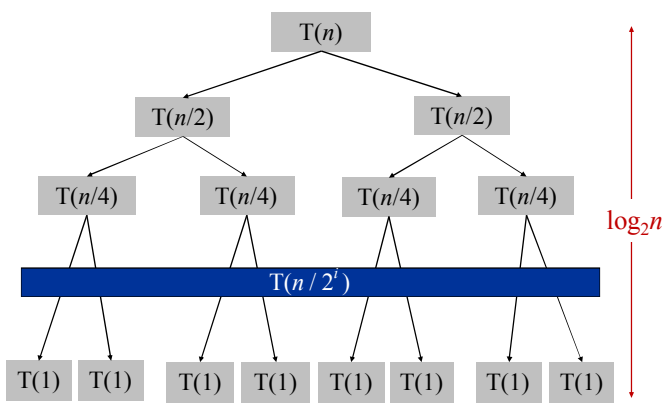
$bn\log_2 n$

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

Solving Recurrences: Recursion Tree

Finding Maximum and Minimum:

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + b & \text{if } n > 1 \end{cases}$$



No. of Nodes at this Level	Time for Each Node	Time at this Level
1	b	$1(b) = b$
2	b	$2(b) = 2b$
4	b	$4(b) = 4b$
2^i	b	$2^i(b) = 2^i b$
n	b	$n(b) = bn$

$$T(n) = b + 2b + 2^2b + \cdots + 2^ib + \cdots + bn = 2bn - 1 = O(n)$$

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

Solving Recurrences: Master Theorem

Solving Recurrences: Master Theorem

- Given: a *divide and conquer* algorithm
  - An algorithm that divides the problem of size n into a subproblems, each of size n/b, where $a \geq 1$, $b > 1$
  - The a subproblems are solved recursively, each in time $T(n/b)$
  - Let the cost of each stage (i.e., the work to divide the problem + combine solved subproblems) be described by the function $f(n)$
  - Let $T(n)$ be defined by the recurrence

    $$T(n) = aT(n/b) + f(n)$$

- Then, the Master Theorem gives us a *cookbook* for the algorithm's running time.

Solving Recurrences: Master Theorem

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined by the recurrence

$$T(n) = aT(n/b) + f(n).$$

Then $T(n)$ has the following asymptotic bounds:

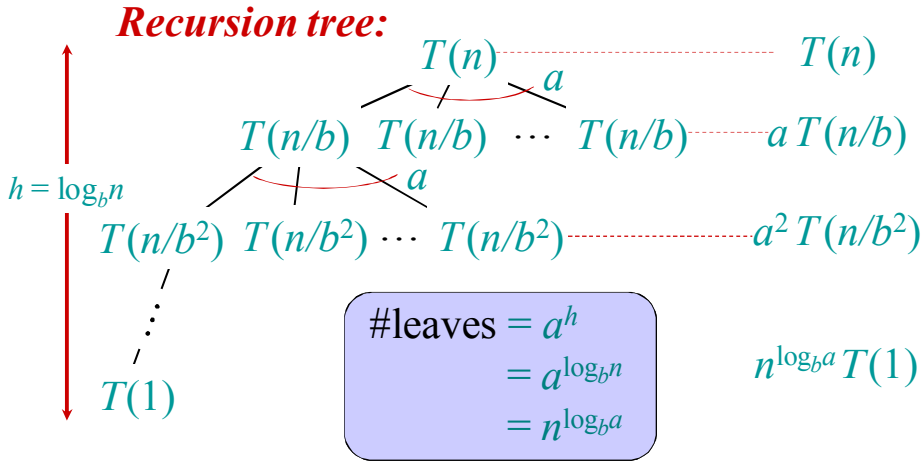
- If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then
  $T(n) = \Theta(n^{\log_b a})$
- If $f(n) = \Theta(n^{\log_b a})$ then
  $T(n) = \Theta(n^{\log_b a} \log n)$
- If $f(n) = O(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$ then
  $T(n) = \Theta(f(n))$

Solving Recurrences: Master Theorem

- Essentially, the Master theorem compares the function $f(n)$ with the function $g(n) = n^{\log_b(a)}$.
  Roughly, the theorem says:
  - If $f(n) << g(n)$ then $T(n) = \Theta(g(n))$
  - If $f(n) \approx g(n)$ then $T(n) = \Theta(g(n) \log_b n)$
  - If $f(n) >> g(n)$ then $T(n) = \Theta(f(n))$
- Now go back and memorize the theorem !!!

Idea of Master Theorem

Recursion tree:



$$\begin{aligned}\#\text{leaves} &= a^h \\ &= a^{\log_b n} \\ &= n^{\log_b a}\end{aligned}$$

Master Theorem: Three Common Cases

Compare $f(n)$ with $n^{\log_b a}$

1. $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$.
   - $f(n)$ grows polynomially slower than $n^{\log_b a}$

   Solution: $T(n) = O(n^{\log_b a})$.

2. $f(n) = \Theta(n^{\log_b a})$
   - $f(n)$ and $n^{\log_b a}$ grow at similar rates.

   Solution: $T(n) = O(n^{\log_b a} \log n)$.

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$.
   - $f(n)$ grows polynomially faster than $n^{\log_b a}$

   Solution: $T(n) = O(f(n))$.

Master Theorem: Examples

Ex. $T(n) = 4T(n/2) + n$

$a = 4,\ b = 2 \qquad \Rightarrow n^{\log_b a} = n^2;\ f(n) = n.$

CASE 1: $f(n) = O(n^{2-\varepsilon})$ for $\varepsilon = 1$.

$\therefore\ T(n) = O(n^2)$.

Ex. $T(n) = 4T(n/2) + n^2$

$a = 4,\ b = 2 \qquad \Rightarrow n^{\log_b a} = n^2;\ f(n) = n^2.$

CASE 2: $f(n) = \Theta(n^2)$.

$\therefore\ T(n) = O(n^2 \log n)$.

Ex. $T(n) = 4T(n/2) + n^3$

$a = 4,\ b = 2 \qquad \Rightarrow n^{\log_b a} = n^2;\ f(n) = n^3.$

CASE 3: $f(n) = \Omega(n^{2+\varepsilon})$ for $\varepsilon = 1$

$\therefore\ T(n) = O(n^3)$.

Master Theorem: Sample Question

Solve the following recurrences by using the Master theorem.

(i) $T(n) = 4T(n/2) + 8n + 5$

(ii) $T(n) = 9T(n/3) + 5n\sqrt{n} + 8\log n$

(iii) $T(n) = 7T(n/4) + 3n^2+4n+5$

(iv) $T(n) = 20T(n/4) + 3n^2+4n+5$

(v) $T(n) = 9T(n/3) + 5n^2+7$

(vi) $T(n) = 6T(n/3) + 3n^2+7n$

(vii) $T(n) = 8T(n/2) + 5n^3+7n^2$